

MODULE 08 · DOMAIN 3 - DESIGN HIGH-PERFORMING ARCHITECTURES

Storage and Database Performance

Matching the store to the access pattern.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 8 of 22

Ask three questions: **who reads it, how do they reach it, and how fast must it be.** The service falls out of the answers.

Storage types, and what each is for

SERVICE	TYPE	CHOOSE IT WHEN
Amazon EBS	Block, one instance	A disk for one instance. Databases
Instance store	Block, physically attached	Temporary scratch. Lost on stop
Amazon EFS	File, NFS, many instances	Shared Linux file system
FSx for Windows	File, SMB	Windows shares with Active Directory
FSx for Lustre	File, high performance	HPC and ML. Links to S3
Amazon S3	Object	Web-accessible objects at any scale

ON THE EXAM

'Shared across multiple instances' means EFS on Linux, FSx for Windows.

EBS volume types

TYPE	CHARACTERISTICS	CHOOSE IT WHEN
gp3	Baseline 3,000 IOPS, throughput set separately	The default for most workloads
gp2	IOPS scale with volume size	Older generation. gp3 is better
io2 Block Express	Highest IOPS, sub-millisecond	Critical databases
st1	Throughput optimised HDD	Big sequential reads. Logs
sc1	Cold HDD, lowest cost	Infrequent sequential data

ON THE EXAM

'IOPS' means gp3 or io2. 'Sequential throughput' means st1. HDD cannot boot.

Relational or NoSQL?

RDS OR AURORA

When relationships matter

- Complex joins and ad-hoc queries
- Transactions across multiple tables
- An existing application expecting SQL
- Schema is known and stable

DYNAMODB

When scale and latency matter

- Single-digit millisecond at any scale
- Access is by key, and the pattern is known
- Enormous or unpredictable request rates
- Serverless, no instance to size

THE TIEBREAKER

'Millions of requests with consistent single-digit millisecond latency' means DynamoDB.
'Complex queries and joins' means relational.

Caching, and where it belongs

- ◆ **ElastiCache for Redis.** Persistence, replication, sorted sets, pub/sub. The default when the question needs durability or advanced structures.
- ◆ **ElastiCache for Memcached.** Simple, multi-threaded, no persistence. For a plain cache that can be lost.
- ◆ **DynamoDB Accelerator.** An in-memory cache specifically for DynamoDB, taking reads from milliseconds to microseconds.
- ◆ **CloudFront.** Caches at the edge, close to users. For content rather than database results.
- ◆ **Session state** in ElastiCache is the standard way to make a web tier stateless.

ON THE EXAM

DAX only ever fronts DynamoDB. If the database is relational, DAX is automatically wrong.

Aurora and RDS specifics

- ◆ **Aurora** is MySQL and PostgreSQL compatible, with storage that grows automatically and six copies across three AZs.
- ◆ **Aurora Serverless** scales capacity automatically. Good for intermittent or unpredictable workloads.
- ◆ **Aurora global database** replicates cross-Region, typically under a second.
- ◆ **RDS Proxy** pools connections. The answer when many Lambda functions exhaust database connections.
- ◆ **Read replicas** can be added to both, and Aurora supports up to fifteen with very low replica lag.

ON THE EXAM

Availability warning: Aurora Serverless v1 reached end of life in March 2025. Version two was renamed simply Aurora Serverless in April 2026.

A content management application runs on six EC2 instances behind a load balancer. Editors upload documents which must be immediately visible to all instances, and any instance may serve any request.

The team currently copies files between instances with a synchronisation script, which frequently leaves instances out of step.

Which storage solution should be used?

- ☐ A An EBS volume attached to each instance, kept in sync by the script
- ☐ B Amazon EFS mounted on all six instances
- ☐ C Instance store volumes on each instance
- ☐ D An EBS Multi-Attach io2 volume shared by all six instances

◆ PAUSE AND DECIDE

Which storage solution should be used?

A Per-instance EBS with sync script

This is the current broken design. Synchronisation always has a window where instances disagree.

C Instance store

Local, not shared, and lost when an instance stops. Worse than the current design in every respect.

B EFS mounted on all instances

One file system mounted by every instance, so a write is immediately visible everywhere. No synchronisation exists to fail.

D EBS Multi-Attach

Real, but limited to io1 and io2 in one AZ, and it needs a cluster-aware file system. Not a general shared-storage answer.

THE SIGNAL

Multiple instances needing the same files at the same time is EFS. It is one of the most repeated patterns on the exam.

A mobile game stores player profiles, accessed by player ID. It has three million daily users and needs consistent read latency under ten milliseconds worldwide. Traffic varies by a factor of twenty between quiet and peak hours.

The team does not want to manage database instances or plan capacity, and there are no reporting or join requirements.

Which database should be used?

- ☐ A Amazon RDS for MySQL with read replicas
- ☐ B Amazon DynamoDB with on-demand capacity
- ☐ C Amazon Redshift
- ☐ D Amazon Aurora PostgreSQL with Auto Scaling replicas

◆ PAUSE AND DECIDE

Which database should be used?

A RDS MySQL with replicas

Instances to size and manage, and a twenty-fold traffic swing means either over-provisioning or throttling.

C Amazon Redshift

A data warehouse for analytical queries over large datasets. Entirely wrong for single-record lookups by key.

B DynamoDB on-demand

Key-based access, single-digit millisecond latency at any scale, no instances to manage, and on-demand absorbs the twenty-fold swing.

D Aurora with Auto Scaling replicas

Closer, and it scales reads well, but you still manage instance classes, and the workload needs none of Aurora's relational strengths.

THE SIGNAL

Access by key, single-digit milliseconds, and no joins is DynamoDB. On-demand capacity is the phrase for unpredictable volume.

A read-heavy product catalogue runs on Amazon RDS. Ninety percent of queries request the same two thousand popular products, and the database instance is at eighty percent CPU during business hours.

The application team can change the code. They want to reduce database load and improve response times without changing the database engine.

What is the MOST effective change?

- ☐ A Add Amazon ElastiCache and cache the popular product queries
- ☐ B Add DynamoDB Accelerator in front of the database
- ☐ C Migrate the catalogue to Amazon Redshift
- ☐ D Enable Multi-AZ so the standby can serve read queries

◆ PAUSE AND DECIDE

What is the MOST effective change?

- A

ElastiCache for popular queries
A small hot subset serving most requests is the ideal caching pattern. Repeat reads are served from memory and never reach the database.
- B

DynamoDB Accelerator
DAX only works with DynamoDB. This is an RDS database, so DAX cannot be placed in front of it at all.
- C

Migrate to Redshift
Redshift is for analytical queries over large datasets, not high-volume lookups of individual products.
- D

Multi-AZ for read scaling
The Multi-AZ standby serves no traffic. This is the classic confusion with read replicas.

THE SIGNAL

Ninety percent of traffic hitting a small subset is the textbook cache signature. And DAX is only ever for DynamoDB.

A machine learning team trains models on a two hundred terabyte dataset held in S3. Training runs across forty GPU instances that all read the same data, and they need aggregate throughput of hundreds of gigabytes per second.

Reading directly from S3 is proving too slow, and copying the data to each instance's EBS volume takes hours.

Which storage solution should be used?

- ☐ A Amazon EFS in General Purpose mode mounted on all instances
- ☐ B Amazon FSx for Lustre linked to the S3 bucket
- ☐ C Provisioned IOPS io2 EBS volumes on each instance
- ☐ D Amazon FSx for Windows File Server

◆ PAUSE AND DECIDE

Which storage solution should be used?

- A

 EFS General Purpose
Shared and scalable, but not built for hundreds of gigabytes per second. It is a general file system, not an HPC one.
- B

 FSx for Lustre linked to S3
Built for exactly this: sub-millisecond latency, hundreds of GB/s, and it presents the S3 bucket as a file system without a copy step.
- C

 io2 volumes per instance
Not shared, and it keeps the hours-long copy step the question complains about.
- D

 FSx for Windows
SMB file shares for Windows workloads. Wrong protocol and wrong performance profile for Linux GPU training.

THE SIGNAL

Hundreds of GB/s, HPC or machine learning, and data already in S3 is the exact signature of FSx for Lustre.

A serverless application uses hundreds of concurrent Lambda functions that each open a connection to an Amazon RDS for PostgreSQL database. During peaks the database rejects connections, and functions fail.

The team wants to keep the relational database and does not want to restructure the application.

Which solution addresses this?

- ☐ A Amazon RDS Proxy in front of the database
- ☐ B Migrate the database to DynamoDB
- ☐ C Add read replicas and distribute connections across them
- ☐ D Increase the RDS instance size to raise the connection limit

◆ PAUSE AND DECIDE

Which solution addresses this?

- A

RDS Proxy
Pools and reuses connections, so hundreds of functions share a small number of real database connections. Built for this exact problem.
- B

Migrate to DynamoDB
Ruled out by the requirement to keep the relational database and avoid restructuring the application.
- C

Read replicas
Adds read capacity but the functions also write, and each replica has its own connection limit. It defers the problem.
- D

Bigger instance
Raises the ceiling at permanent cost, and Lambda concurrency can scale past any fixed ceiling you buy.

THE SIGNAL

Lambda plus a relational database plus connection exhaustion is always RDS Proxy. Compare this with reserved concurrency in module seven.

A video rendering workload writes large temporary files during processing and reads them back within the same job. The files are discarded when each job finishes. Jobs run on r5d instances, which include local NVMe storage.

The team currently writes these temporary files to a Provisioned IOPS EBS volume and finds it both slow and expensive.

What should the solutions architect recommend?

- ☐ A Write temporary files to the instance store volumes
- ☐ B Increase the provisioned IOPS on the EBS volume
- ☐ C Write temporary files to Amazon EFS
- ☐ D Write temporary files to S3 with Transfer Acceleration

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Instance store
Physically attached NVMe gives the highest IOPS available, costs nothing extra, and its volatility does not matter for files that are discarded anyway.
- B** More provisioned IOPS
Increases both performance and cost, when local storage already included in the instance price is faster.
- C** EFS for temporary files
A network file system is slower than local disk and adds cost. Nothing here needs sharing between instances.
- D** S3 with Transfer Acceleration
Object storage for scratch files within a job, and Transfer Acceleration is for long-distance uploads, not local performance.

THE SIGNAL

Instance store's weakness, losing data on stop, is irrelevant when the data is temporary. That is the whole question.

An analytics team needs to run ad-hoc SQL queries over twelve terabytes of compressed log files stored in S3. Queries run a few times a week and each scans a small portion of the data. The team wants to avoid running any infrastructure and wants to pay only for the queries they run.

Which solution is MOST cost-effective?

- ☐ A Load the data into Amazon Redshift and query it there
- ☐ B Query the data in place with Amazon Athena
- ☐ C Launch an EMR cluster and query with Spark SQL
- ☐ D Load the data into an RDS instance and query with SQL

◆ PAUSE AND DECIDE

Which solution is MOST cost-effective?

A Load into Redshift

Excellent for frequent, complex analytics, but it means running a cluster continuously for queries used a few times a week.

C EMR cluster

Powerful and more operational work than either alternative, with cluster time to pay for and manage.

B Athena on S3

Serverless, queries data where it already sits, and charges per terabyte scanned. A few queries a week costs very little.

D Load into RDS

A transactional database is the wrong tool for twelve terabytes of log analytics, and the load process alone is substantial.

THE SIGNAL

Infrequent, ad-hoc, data already in S3, pay per query. That combination is Athena. Frequent complex analytics would flip it to Redshift.

A DynamoDB table storing sensor readings is experiencing throttling. Investigation shows that all writes use the current date as the partition key, so on any given day every write goes to the same partition.

The table's provisioned capacity is well above the total request rate, yet throttling persists.

What is the MOST appropriate fix?

- ☐ A Increase the table's provisioned write capacity further
- ☐ B Change the partition key to one with high cardinality, such as sensor ID
- ☐ C Add a DynamoDB Accelerator cluster in front of the table
- ☐ D Enable DynamoDB Streams to spread writes over time

◆ PAUSE AND DECIDE

What is the MOST appropriate fix?

- A

 More provisioned capacity
Capacity is already ample. Adding more does not help when one partition is the bottleneck; it just costs more.
- B

 High-cardinality partition key
Spreads writes evenly across many partitions, so the table's capacity is actually usable rather than concentrated on one hot partition.
- C

 DAX in front of the table
DAX is a read cache. It does nothing for write throttling.
- D

 DynamoDB Streams
Streams capture a change log for downstream processing. They do not redistribute or delay writes.

THE SIGNAL

Throttling with spare capacity always means a hot partition. The fix is the key design, never more capacity.

Module 8 takeaways

- ◆ **Shared across instances** means EFS on Linux, FSx for Windows, FSx for Lustre for extreme throughput.
- ◆ **A small hot subset of data** means a cache. DAX only ever fronts DynamoDB.
- ◆ **Lambda exhausting database connections** means RDS Proxy.
- ◆ **Temporary, discarded data** makes instance store the fast and free answer.
- ◆ **Throttling with spare capacity** is a hot partition. Fix the key, not the capacity.

ON THE EXAM

Next: getting data in, and getting content out, at scale and at distance.